

SHARIF UNIVERSITY OF TECHNOLOGY

Department of Electrical Engineering

PROJECT ASSIGNMENT

Cache-Aware SIMD and Strassen Matrix Multiplication

INSTRUCTOR

Not identified in the supplied materials

Contents

1 Objective and mathematical contract 2

2 Direct multiplication and input generation 2

3 Memory-layout transformation 2

4 SIMD leaf kernel 2

5 Recursive multiplication 2

6 Correctness tests 2

7 Timing experiment 3

8 Deliverables 3

1 Objective and mathematical contract

Implement and evaluate multiplication of two square double-precision matrices. Provide a direct i-j-k reference implementation and a hybrid multiplication using a transposed second operand, a four-by-four SIMD leaf and seven-product Strassen recursion. For every logical element, compute C[i,j]= sum_k A[i,k]*B[k,j]. Inputs and output must occupy separate storage. Use row-major matrices and support independent input and output leading dimensions.

2 Direct multiplication and input generation

Implement a pointer-based scalar reference that accumulates one output dot product at a time. Preserve this straightforward mathematical reference when implementing optimizations. Generate reproducible test inputs with an explicit nonzero seed. Provide zero, identity, signed and positive rational input patterns. Check input dimensions, allocation-size overflow and allocation failure before computation. Use double precision consistently for data and accumulation.

3 Memory-layout transformation

Allocate 64-byte-aligned storage and transpose B into a packed leaf buffer so that both vectors in an inner product are read sequentially. Define the transformation B_T[j*n+k]=B[k*s_B+j]. Free the transpose buffer after the product. Do not assume every submatrix row or offset is vector-aligned. Include transpose and temporary-allocation costs in optimized timing. Explain the access pattern without claiming a measured cache-miss reduction when no cache counters have been collected.

4 SIMD leaf kernel

Evaluate up to sixteen output scalars for each four-by-four tile. Use four-double AVX vectors to reuse loaded operand vectors across the tile. Sum accumulator lanes and handle remaining inner-product terms scalarly. Include partial row and column tiles and dimensions below four. Provide runtime AVX selection and a scalar fallback; a scalar-only build must also pass the numerical tests. SIMD instructions must not be executed on an unsupported processor.

5 Recursive multiplication

Use a default leaf cutoff of 512, with a command-line option to select another positive cutoff. Split even dimensions above the cutoff into four equal quadrant views. Compute all seven Strassen products and the four recombination expressions. Pass logical dimension and leading dimensions as separate arguments. Use packed intermediate products and temporary sums while retaining parent strides for quadrant views. Compute odd subproblems with the complete leaf routine rather than omitting a row or column. Release every temporary allocation on success or failure.

6 Correctness tests

Compare all output elements to the direct reference using a finite absolute-plus-relative bound. A zero reference must be handled without division by zero, and a NaN or infinity must fail verification. Test dimensions 1,2,3,4,5,7,8,12,16,31,32,33 and 64 with zero, identity-left, identity-right and signed operand patterns. Use a cutoff of four to exercise recursion. Test distinct padded leading dimensions and verify that output-row padding is unchanged. Run the same suite with AVX enabled when available and with SIMD disabled.

7 Timing experiment

Measure dimensions 64,128,256,512 and 1024, using seed 42 and cutoff 512 for three repetitions each. Time the direct and optimized products separately. State whether timing is wall-clock or CPU time and identify the compiler, optimization level, processor and SIMD state. Initialization of the main matrices may occur before timing, but preparation internal to the optimized product must be included. Record every repetition in CSV and verify every timed output.

Report median elapsed times, optimized-time range, and the median of per-repetition speedup ratios. Distinguish direct SIMD-leaf experiments from experiments that actually enter Strassen recursion. Report error values together with the comparison tolerances. Discuss the limited number of repetitions, execution-order effects and the absence of a tuned-library comparison or isolated optimization ablations.

8 Deliverables

Supply the matrix-library source and header, benchmark entry point, deterministic test source, Windows and POSIX build instructions, and measured CSV. The technical report must explain all seven product identities, output recombination, row-stride handling, SIMD tile scheduling, fallback and tail cases, allocation lifecycle, numerical criteria, theoretical work/storage and the measured results. Keep source and execution instructions in the code directory and the complete report in the report directory.